

AI Robot Arm

Meet the AI-powered robotic hand that watches your gestures and copies them in real time no controller required

What if a robot could simply watch your hand and do exactly what you do? No joystick, no buttons, no complicated input device just your fingers moving naturally in front of a webcam. That's the idea behind this project: an AI-powered robotic hand that recognises human hand gestures through a camera and reproduces them physically using servo motors, machine learning, and an Arduino microcontroller.

It sounds like science fiction, but the whole system runs on a standard laptop. And that, it turns out, is kind of the point.

A problem worth solving

The inspiration behind this build goes deeper than building something impressive. Today's bionic and prosthetic hands are extraordinarily expensive, have limited service lifespans, and cost even more to repair or upgrade. For many people who need them most, they're simply out of reach.

This project challenges that reality. Because the system runs on any standard computer and uses affordable, widely available components, it drastically lowers the barrier to gesture-controlled robotics. The applications stretch well beyond the workshop: assistive technology for people with limited mobility, remote exploration of hazardous or inaccessible environments, rehabilitation tools, and educational robotics platforms are all realistic use cases for a system like this.

The ambition isn't to replace professional medical devices overnight it's to prove that the core technology can be made accessible, trainable, and affordable.

How the system works

The pipeline from human gesture to robot movement is surprisingly elegant. A laptop webcam captures a live video feed of the user's hand. Google's MediaPipe library then takes over, detecting the hand within each frame and mapping 21 landmark points across it fingertips, knuckle joints, the base of the palm, and every segment in between. These points give the system a precise, real-time skeleton of the hand.

From those 21 coordinates, the software can determine which fingers are extended, which are curled, and roughly what angle each joint is at. This landmark data is then normalised adjusted so that hand size and distance from the camera don't affect the results and passed to a machine learning classifier. Two approaches were tested during development: a K-Nearest Neighbours (KNN) model built with scikit-learn, and a Convolutional Neural Network (CNN) trained using TensorFlow. Both were evaluated on a custom dataset of hand gestures captured manually under a range of lighting conditions. Once the model identifies the gesture, Python sends a serial command via USB to an Arduino microcontroller. The Arduino then drives a set of MG996R servo motors — one per finger — which physically move the joints of the robotic hand to match the detected gesture.

The whole loop — from hand movement to robot response — happens fast enough to feel immediate.

Teaching it your own gestures

One of the most genuinely useful features of this build is that the gesture vocabulary isn't fixed. The system is designed so that you can record your own gestures, label them with whatever name makes sense to you, and retrain the model yourself. Want the hand to recognise a specific sign language alphabet? A set of custom control commands? Gestures meaningful to a particular user's needs? All of that is possible.

This makes the system far more practical for real-world assistive applications, where one-size-fits-all solutions rarely work. A person might need very specific gestures recognised reliably and being able to personalise the training data without specialist knowledge is a significant advantage.

Building the dataset

Creating a reliable training dataset turned out to be one of the more painstaking parts of the project. Gesture samples were collected manually across different sessions and lighting conditions — bright rooms, darker environments, different camera angles to make the model robust rather than brittle.

Each sample went through preprocessing: landmark extraction, coordinate normalisation, and filtering to remove blurry frames, partially visible hands, or invalid detections. Getting this data cleaning step right had a measurable impact on the final accuracy. Garbage in, garbage out — a lesson that applies just as much in robotics as anywhere else in machine learning.

The final model achieved approximately 97% gesture recognition accuracy, with very low response latency under good lighting conditions.

The tricky bits

No build goes entirely smoothly, and this one was no exception. The first major headache came during setup: getting MediaPipe to cooperate with newer versions of Python proved genuinely difficult. The library and the cvzone helper package simply refused to work together on recent Python releases. Even rolling back to older versions didn't immediately solve the problem — an alternative installation method ultimately had to be found before things started working properly.

Latency was the other significant challenge. A robotic hand that lags noticeably behind your movements quickly becomes frustrating and impractical. Considerable effort went into optimising the preprocessing pipeline — reducing the time between gesture detection and the serial command being sent to the Arduino. Lighting conditions also turned out to matter more than expected: inconsistent or low light confused the landmark detection and degraded recognition quality, which led to additional testing across different environments.

Synchronising the AI predictions with the physical hardware — making sure that a recognised gesture translated into smooth, reliable motor movement rather than a jerky or delayed response — was one of the most technically interesting problems to solve.

The hardware

The physical build centres on MG996R servo motors, chosen for their torque and reliability. Each motor controls a different finger or joint of the robotic hand. The Arduino connects to a Sensor Shield v5.0, which made wiring significantly cleaner and more reliable. Instead of routing every servo signal wire directly to individual Arduino pins, the shield provides dedicated connectors for each servo power, ground, and signal in one organised board. This was especially useful when managing five servo motors simultaneously, keeping the build tidy and reducing the risk of loose connections during movement.

Assembling the hand itself required as much engineering thinking as programming: managing the power supply across multiple motors, and ensuring the mechanical structure was stable enough to move repeatably without flexing or slipping. Getting the mapping between gesture classifications and motor positions right so that each recognised gesture produced the correct finger configuration involved a fair amount of calibration and iteration.

You'll need:

Arduino microcontroller with Sensor Shield v5.0

MG996R servo motors × 5 (one per finger)

Laptop with webcam

Python with MediaPipe and OpenCV installed

TensorFlow and scikit-learn (for model training)

Jumper wires and appropriate power supply

Robotic hand frame (3D printed or assembled)

USB cable for serial communication

What comes next

The current prototype reliably reproduces gestures under good conditions, and the 97% accuracy figure holds up well in practice. But there's plenty more to build. Plans include expanding the gesture dataset substantially, improving the CNN architecture, and adding continuous gesture tracking so the hand responds to fluid movement rather than discrete, held poses.

A mobile application for remote monitoring and control is also on the roadmap, along with improvements to the mechanical stability of the hand itself. Longer term, the aim is to explore applications in healthcare, assistive robotics, and remote interaction systems areas where affordable, trainable gesture control could make a real difference.

For now, though, the project already demonstrates something worth paying attention to: that meaningful, intelligent robotics doesn't require a large budget or a research lab. Just curiosity, persistence, Python, and a willingness to debug serial communication until it works